

P3540R2

#embed offset parameter

Published Proposal, 2025-06-05

Authors:

[JeanHeyd Meneide](#)

[Shepherd \(Shepherd's Oasis LLC\)](#)

Latest:

https://thephd.dev/_vendor/future_cxx/papers/d3540.html

Paper Source:

[GitHub ThePhD/future_cxx](#)

Implementation:

[GitHub ThePhD/embed](#)

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

EWG

Abstract

An additional, user-supported embed parameter implemented in Clang and GCC for providing an offset.

Table of Contents

1 Changelog

1.1 Revision 2 - June ^{5th}, 2025

1.2 Revision 1 - February 14th, 2025

1.3 Revision 0 - December 13th, 2024

2 Introduction and Motivation

3 Design

4 Wording

4.1 Intent

4.2 Proposed Language Wording

4.2.1 Add to the *control-line* production in §15.1 Preamble [**cpp.pre**] a new grammar production for **offset**

4.2.2 Add a new sub-clause §15.4.2.✨ under Resource Inclusion for Embed parameters for the new **offset** parameter [**cpp.embed.param.offset**]

4.3 Add a new example to the **if_empty** embed parameter [cpp.embed.if.empty] section

§ 1. Changelog

§ 1.1. Revision 2 - June 5th, 2025

- Design was expanded upon in Revision 1 and further explicated in this version to prevent people from being confused about what is being proposed here and what the specification was asking for. It was not recorded in the previous changelog: it is being recorded here.
- Revision 2 does not add or change any of the wording in a way that matters; it just fixes clerical errors, typos, and aligns dates.
- One such typo: fixes "It shall provide" to "It provides", as the "shall" is spurious in C++ wording since it does not provide anything.
- §3 Design elaborates that this is meant to be that **offset** applies before **limit** is applied, as what is implemented in **clang::offset** and **gnu::offset** today.
- Some discussion during an SG22 meeting about restricting the order of embed parameters. This isn't present in any implementation or in the C wording, and it's an awful experience overall: rejected this design to proceed forward.

§ 1.2. Revision 1 - February 14th, 2025

- Minor typo and wording cleanup.

§ 1.3. Revision 0 - December 13th, 2024

- Initial release.

§ 2. Introduction and Motivation

The goal is to add the extremely-popular and already-implemented `gnu::offset` and `clang::offset` parameters as standard parameters. That is the only motivation of this proposal; to standardize existing practice.

Originally, users asked to add this parameter, but only after C23 standardized. Given the late stage that users have asked -- waiting until the very end -- it has to be added separately. This proposal aims to standardize what users have asked for, and what Clang and GCC have implemented.

§ 3. Design

The design of `offset (some-preprocessor-constant-value)` is straightforward:

- `offset` discards `some-preprocessor-constant-value` items, up to the full size of the resource;
- nothing happens if `some-preprocessor-constant-value` evaluates to `0`;
- if `offset` is greater than or equal to the size of the resource, the resource is considered empty;
- `limit` does not shrink the file to a given size before `offset`; `offset` is applied to the original file size (called the "implementation-resource-count" in the wording).

These are the only tenets of the design, and match the practice for existing implementations `gnu::offset` and `clang::offset`. It is also how the original author envisioned this when it was first PR'd to Clang, and the original tests (plus new ones) still pass in the LLVM/clang and gnu/gcc repositories.

There was some discussion during the June 4th, 2025 about the order of parameters. This turned into a significant enough change that it is a separate paper. No additional discussion or changes are proposed for this paper: it is exactly a standardization of existing practice and what was approved in Austria.

§ 4. Wording

This wording is relative to C++'s latest working draft.

§ 4.1. Intent

The intent of the wording is to provide a preprocessing directive that:

- allows an `offset` parameter skipping over a number of elements before beginning to write out elements to the comma-delimited list;
- and, applies to the actual size of the resource (and not to any applied `limit`).

§ 4.2. Proposed Language Wording

§ 4.2.1. Add to the *control-line* production in §15.1 Preamble [cpp.pre] a new grammar production for `offset`

embed-standard-parameter:

limit (*pp-balanced-token-seq*)

offset (*pp-balanced-token-seq*)

prefix (*pp-balanced-token-seq_{opt}*)

suffix (*pp-balanced-token-seq_{opt}*)

if_empty (*pp-balanced-token-seq_{opt}*)

§ 4.2.2. Add a new sub-clause §15.4.2.✧ under Resource Inclusion for Embed parameters for the new **offset** parameter [cpp.embed.param.offset]

15.4.2.✧ **offset parameter** [cpp.embed.param.offset]

- 1 An *embed-parameter* of the form **offset** (*pp-balanced-token-seq*) denotes the number of elements to be skipped from the resource. It shall appear at most once in the *embed-parameter-seq*.
- 2 The *pp-balanced-token-seq* is evaluated as a *constant-expression* using the rules as described in conditional inclusion ([cpp.cond]), but without being processed as in normal text an additional time.
- 3 The *constant-expression* shall be an integral constant expression whose value is greater than or equal to zero. It provides the value for *resource-offset*. The *embed* directive performs *resource-offset* consecutive calls to **std::fgetc** ([cstdi.syn]) from the resource, as a binary file. If a call to **std::fgetc** returns **EOF**, the resource is considered empty. Otherwise, the result of the call is discarded. The resource-count is changed to be
 - the value is computed using the **limit embed-parameter** ([cpp.embed.param.limit]), if the **limit embed-parameter** is present. Let the value computed by the **limit embed-parameter** be *limit-value*. resource-count becomes

.
 - Otherwise,

.

[Example:

```
constexpr const unsigned char sound_signature[] = {  
    // a hypothetical resource capable of expanding to four or more elements  
    #embed <sdk/jump.wav> limit(2+2)  
};  
  
constexpr const unsigned char truncated_sound_signature[] = {  
    // the same hypothetical resource capable of expanding to four or more el  
    #embed <sdk/jump.wav> offset(2) limit(2)  
};  
  
static assert(sizeof(sound_signature) == 4);  
static assert(sizeof(truncated_sound_signature) == 2);  
static assert(sound_signature[2] == truncated_sound_signature[0]);  
static assert(sound_signature[3] == truncated_sound_signature[1]);
```

– end example]

§ 4.3. Add a new example to the `if_empty` embed parameter [cpp.embed.if.empty] section

[Example: Given a resource <single_byte> that has an implementation-resource-count of 1, the following directives:

```
#embed <single_byte> offset(1) if_empty(44203)  
#embed <single_byte> limit(0) offset(1) if_empty(44203)
```

are replaced with:

```
42203  
42203
```

– end example]

[Example: Given a resource <single_byte> that has an implementation-resource-count of 1, `has_embed` will be considered empty despite `limit(1)`, as `offset(1)` has exhausted the implementation-resource-count:

```
int infinity_zero () {  
#if __has_embed(<single_byte> limit(1) offset(1) prefix(some tokens)) ==  
__  
    // if <single_byte> exists, this  
    // conditional inclusion branch is taken and the function  
    // returns 0.  
    return 0;  
#else  
    // otherwise, the resource does not exist  
#error "The resource does not exist"  
#endif  
}
```

– end example]